

DESENVOLVIMENTO E COMPARAÇÃO DE ARQUITETURAS DE APRENDIZADO PROFUNDO APLICADAS AO XADREZ

Daniel Couto *Aprendizado de Máquina 2 IMPA 6 de Julho de 2026 -*

1. INTRODUÇÃO E MODELAGEM DO ESTADO

Este trabalho aborda o desenvolvimento e a avaliação comparativa de duas abordagens de Aprendizado Profundo aplicadas ao jogo de xadrez: um modelo baseado em Aprendizado por Reforço por meio de redes Q-Profundas (Deep Q-Network, ou DQN) e um modelo de Aprendizado Supervisionado baseado em Redes Residuais (ResNet) para regressão de valor.

Ambas as redes compartilham a mesma representação de entrada para garantir a consistência das comparações. O estado do tabuleiro é mapeado para um tensor tridimensional com dimensões 15 x 8 x 8. As 12 primeiras camadas representam os tipos de peças binárias indexadas por cor e tipo (peões, cavalos, bispos, torres, rainhas e reis, mapeados separadamente para brancas e pretas). As camadas 13 e 14 codificam as informações sobre os direitos de roque das brancas e das pretas, respectivamente. A camada 15 representa a vez de quem vai jogar, em formato binário, conforme implementado no ambiente computacional. Esta codificação preserva a geometria bidimensional original do tabuleiro de xadrez, em formato 8 x 8, em todas as fatias de informação.

2. ARQUITETURA DOS MODELOS E DETALHES DE TREINAMENTO

2.1. Abordagem por Aprendizado Supervisionado (ResNet)

O modelo ChessEvalResNet atua como uma função de avaliação heurística de posições, estimando o valor da vantagem em centipawns (cp).

- **Dataset:** Dados originários da plataforma Kaggle (Chess Evaluations), compostos por mais de 12 milhões de posições rotuladas por análises matemáticas extraídas do motor Stockfish em alta profundidade de busca.
- **Arquitetura:** O modelo inicia com um bloco preparatório (*Stem*) composto por uma camada convolucional 3x3 que projeta os 15 canais de entrada para uma dimensão interna de 64 canais, seguida por Normalização em Lote (*BatchNorm2d*) e ativação ReLU. Na sequência, são empilhados quatro blocos residuais (*ResidualBlock*). Cada bloco possui duas convoluções 3x3 internas com preservação estrita de resolução espacial (8 x 8) via uso de padding igual a 1 e stride igual a 1.

A arquitetura utiliza conexões de salto (*skip connections*) para aplicar a operação matemática $x + F(x)$. Nos blocos 1 e 3, em que ocorre o aumento do número de canais (de 64 para 128, e de 128 para 256, respectivamente), a conexão de identidade é substituída por uma Convolução 1x1 de projeção, ajustando as dimensões de profundidade para permitir o somatório vetorial. Ao final da etapa convolucional, um mecanismo de *Global Average Pooling* (GAP) calcula a média aritmética de cada canal, transformando o tensor tridimensional de dimensão $(256, 8, 8)$ em um vetor de 256 elementos. A cabeça de regressão é uma rede MLP com dimensões $256 \times 256 \times 128 \times 1$, utilizando Dropout de 30% nas camadas intermediárias para regularização. O total de parâmetros treináveis é de 1.626.177.

- **Função de Perda:** Erro Quadrático Médio (MSE).
- **Métricas Obtidas:** O gráfico de aprendizado demonstra estabilização da perda de validação com convergência assintótica. O comportamento indica retenção da capacidade geral de cálculo posicional sem divergência por sobreajuste (*overfitting*).

2.2. Abordagem por Aprendizado por Reforço (DQN)

O modelo DQN foi projetado para aprender uma política de seleção de movimentos por meio da Equação de Bellman.

- **Algoritmo e Otimização:** Implementação de um mecanismo de *Replay Buffer* com estrutura de fila adaptada ao domínio do xadrez, armazenando transições na forma (s_t, a_t, r_t, s_{t+1}) para amostragem aleatória em minibatches, com o objetivo de eliminar a correlação temporal entre lances consecutivos. A taxa de exploração seguiu um decaimento linear ao longo dos episódios gerados pelo método *epsilon-greedy*.
- **Estrutura de Monitoramento:** Métricas registradas via TensorBoard acompanhando a evolução cumulativa da perda (*loss*), tamanho do buffer de transições, passos globais de otimização (*global step*) e o número de lances por partida (*plies*).
- **Comportamento de Treino:** A estabilização do tamanho do buffer correlaciona-se empiricamente com o aumento da média de lances por partida à medida que o fator de exploração reduz, evidenciando o desenvolvimento de políticas de defesa que prolongam a partida.

3. PROTOCOLO DE INTEGRAÇÃO COM A API DO LICHESS

A engine desenvolvida foi integrada ao ecossistema do servidor de xadrez Lichess utilizando a biblioteca de código aberto lichess-bot. A arquitetura de comunicação opera através do protocolo padrão UCI (Universal Chess Interface).

A lógica de decisão em tempo real foi estruturada através de um algoritmo de busca em árvore Minimax com Poda Alfa-Beta acoplado à rede neural supervisionada. O fluxo de execução segue os passos:

1. **Captura e Conversão:** A API do Lichess fornece a string FEN (Forsyth-Edwards Notation) da posição atual do tabuleiro. O interpretador traduz a string para o formato do tensor .
2. **Busca e Poda:** O algoritmo Minimax projeta a árvore de variantes até uma profundidade determinada . A ResNet atua nas folhas avaliando a pontuação em centipawns. A poda Alfa-Beta descarta ramos matematicamente inferiores, reduzindo o custo computacional de $O(b^d)$ para aproximadamente $O(b^{\{d/2\}})$.
3. **Output:** O movimento ótimo selecionado é codificado em formato UCI (ex: e2e4) e enviado de volta via requisição HTTP POST para a API do Lichess.

4. ANÁLISE COMPARATIVA E RESULTADOS

Os modelos foram submetidos a um ambiente de teste estático e dinâmico para avaliar o desempenho empírico fora do conjunto de treino.

4.1. Erro de Predição por Fase do Jogo (ResNet)

A tabela a seguir apresenta o comportamento do Erro Absoluto Médio (MAE) da ResNet medido em centipawns em relação ao número total de peças ativas no tabuleiro:

Fase do Jogo	Número de Peças	MAE Médio (cp)
Abertura	32 a 24 peças	326
Meio-Jogo	23 a 12 peças	545
Finais	Menos de 12 peças	1169

●

Análise: O erro de predição aumenta à medida que o tabuleiro se esvazia (Finais). Este resultado decorre do fato de que as redes neurais convolucionais focam em padrões espaciais locais devido ao tamanho reduzido de seus filtros, falhando em capturar regras

globais de longo alcance de finais de xadrez, onde variações de uma única casa de distância alteram o resultado teórico da posição.

4.2. Confronto Direto (DQN vs. ResNet + Minimax)

Foi conduzido um torneio de 50 partidas controladas com alternância de cores para avaliar a eficácia prática das duas formulações:

- **Vitórias da ResNet (+ Minimax profundidade 2):** 10 partidas (80%)
- **Vitórias do DQN:** 0 partidas (0%)
- **Empates:** 2 partidas (20%)
- **Análise:** O modelo supervisionado superou o agente de aprendizado por reforço. O desempenho justifica-se pelo acoplamento da ResNet ao algoritmo de busca prospectiva (*look-ahead*) fornecido pelo Minimax, mitigando erros táticos imediatos de curto alcance. O DQN demonstrou limitações associadas à esparsidade de recompensa inerente ao xadrez, resultando em instabilidade tática em posições complexas de meio-jogo.

5. CONSIDERAÇÕES FINAIS

Os dados experimentais validam que para o desenvolvimento de engines de xadrez sob restrições de tempo de computação, modelos supervisionados baseados em regressão de valor de datasets consolidados geram heurísticas mais estáveis do que agentes de RL treinados a partir do zero. As limitações da ResNet em finais apontam para a necessidade de integração de bases de dados estáticas de finais (Tablebases) para suprir a perda de precisão espacial em tabuleiros com poucas peças.